# Reviewer Pack — Minimal Order of Quaternionic K-theory and SAT Clause Subset…

Entry 6792bfc47aa2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 20:35:36 UTC

## Minimal Order of Quaternionic K-theory and SAT Clause Subset Entropy Correlation

**Entry ID**: `6792bfc47aa2` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 20:35:36 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebra (Quaternionic K-theory) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subsets)

**Statement**:

> For every instance φ of the Boolean satisfiability problem, the minimal order of the quaternionic K-theory group associated with φ's clause set is linearly correlated with its clause subset entropy such that ord(K_φ) = Θ(entropy(φ)).

**Rationale (proposer's reasoning)**:

> Quaternionic K-theory provides a refined algebraic invariant for studying the structure of algebras, and its minimal order has been shown to be useful in other areas of mathematics. Correlating this with clause subset entropy could expose new structural insights into SAT instances, potentially leading to new complexity separations.

**Taxonomy category**: `Quaternionic_K_theory` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `fbb556b4281631e6`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The conjecture is supported if, after analyzing at least 30 random seeds, the linear regression coefficient $R^2$ is ≥ 0.9 AND the p-value of the correlation test is ≤ 0.05. The conjecture is falsified if either condition is not met.

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | LLM₁ | LLM₂ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 1.00 | SAFE | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - 'Quaternionic K-theory' AND 'Boolean Satisfiability Problem' AND 'minimal order' - 'SAT clause subsets' AND 'quaternionic K-theory' AND 'correlation' - 'entropy of SAT clause subsets' AND 'quaternionic K-theory group'

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + A[i:].index(max(abs(row[i]) for row in A[i:]))
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                continue
            denom = A[i][i]
            for j in range(n):
                A[i][j] /= denom
            for k in range(m):
                if k != i and A[k][i] != 0:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B[0]), len(B)
        C = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                for k in range(p):
                    C[i][j] += A[i][k] * B[k][j]
        return C
```

```
def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if n == 1:
        return A[0][0]
    det = Fraction(0)
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1) ** j * A[0][j] * determinant(submatrix)
    return det

def quaternionic_k_theory_order(clauses):
    n = len(clauses)
    A = [[0] * (n + 1) for _ in range(n + 1)]
    for i, clause in enumerate(clauses):
        for literal in clause:
            if literal > 0:
                A[i][literal - 1] += 1
            else:
                A[i][-1] += 1
    A[-1][-1] = n
    rank = sum(1 for row in gaussian_elimination(A) if any(row))
    return rank

def clause_subset_entropy(clauses):
    total_clauses = len(clauses)
    entropy = 0
    for i in range(1, total_clauses + 1):
        for subset in itertools.combinations(clauses, i):
            prob = Fraction(i, total_clauses)
            entropy -= prob * math.log2(prob)
    return entropy

def generate_random_cnf(n):
    clauses = []
    for _ in range(n):
        clause = random.sample(range(1, 2*n+1), random.randint(1, n))
        clause += [-l for l in clause]
        clauses.append(clause)
    return clauses

n_max = 40
instances_tested = 30
metric_values = []

for _ in range(instances_tested):
    n = random.choice([5, 10, 15, 20, 30, 40])
    clauses = generate_random_cnf(n)
    ord_K_phi = quaternionic_k_theory_order(clauses)
    entropy_phi = clause_subset_entropy(clauses)
    metric_values.append((ord_K_phi, entropy_phi))

if not metric_values:
    return {
        "metric_name": "ord(K_φ) vs entropy(φ)",
```

```
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "empty_metric_values"
        }

    x = [x for x, _ in metric_values]
    y = [y for _, y in metric_values]
    mean_x = sum(x) / len(x)
    mean_y = sum(y) / len(y)
    var_x = sum((xi - mean_x) ** 2 for xi in x) / len(x)
    var_y = sum((yi - mean_y) ** 2 for yi in y) / len(y)
    cov_xy = sum((xi - mean_x) * (yi - mean_y) for xi, yi in zip(x, y)) / len(x)
    r_squared = cov_xy ** 2 / (var_x * var_y)

    return {
        "metric_name": "ord(K_φ) vs entropy(φ)",
        "metric_value": r_squared,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": r_squared >= 0.9 and p_value <= 0.05,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_r_squared = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_r_squared} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_r_squared} std=0.0 support_fraction={support_fractior
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conje
        print(f"RESULT: FALSIFIED counterexample='r_squared_too_low' first_failing_seed={first_fai
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d839372.py", line 134, in <module>
```

```
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d839372.py", line 96, in run_trial
    ord_K_phi = quaternionic_k_theory_order(clauses)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d839372.py", line 65, in quaternionic_
    A[i][literal - 1] += 1
    ~~~~^^^^^^^^^^^^^
IndexError: list index out of range
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> The test crashed before producing data, which means that neither the $R^2$ value nor the p-value could be computed to evaluate the conjecture. | next: Re-run the test with a different seed or investigate the cause of the crash to ensure it can complete without errors.

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 13649 |
| 2 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 9433 |
| 3 | novelty | ollama_remote | glm4:latest | 0 | 0 | 8652 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 8654 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 14994 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 11912 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 7150 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 15636 |
| 9 | judge | ollama_remote | glm4:latest | 0 | 0 | 13044 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 103122 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/6792bfc47aa2.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6792bfc47aa2.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/6792bfc47aa2.tar.gz` (if generated)